Activity: Decision Trees

Math 437

Data

Decision trees can be used for both regression and classification problems. For regression, we'll continue to use our Lahman baseball salary data. For classification, we'll use the RateMyProfessor data.

```
library(Lahman)
library(rpart) # don't think we need this, but it's where decision tree algorithms are
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.1     v stringr   1.5.1
v ggplot2   4.0.1     v tibble    3.2.1
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.0.4
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library(tidymodels)
```

```
-- Attaching packages --------------------------------------- tidymodels 1.3.0 --
v broom        1.0.11     v rsample      1.3.0
v dials        1.4.2      v tune         1.3.0
v infer        1.1.0      v workflows    1.2.0
v modeldata    1.5.1      v workflowsets 1.1.0
v parsnip      1.4.1      v yardstick    1.3.2
```

```
v recipes      1.2.1
-- Conflicts ---------------------------------------- tidymodels_conflicts() --
x scales::discard() masks purrr::discard()
x dplyr::filter()   masks stats::filter()
x recipes::fixed()  masks stringr::fixed()
x dplyr::lag()      masks stats::lag()
x dials::prune()    masks rpart::prune()
x yardstick::spec() masks readr::spec()
x recipes::step()   masks stats::step()
```

```
salaries <- Salaries %>%
  dplyr::select(playerID, yearID, teamID, salary)
peopleInfo <- People %>%
  dplyr::select(playerID, birthYear, birthMonth, nameLast,
         nameFirst, bats)
batting <- battingStats() %>%
  left_join(salaries,
            by =c("playerID", "yearID", "teamID")) %>%
  left_join(peopleInfo, by = "playerID") %>%
  mutate(age = yearID - birthYear -
           1L *(birthMonth >= 10)) %>%
  arrange(playerID, yearID, stint)

batting_2016 <- batting %>% filter(yearID == 2016,
                                   !is.na(salary),
                                   G >= 100, AB >= 200
                                   ) %>%
  mutate(salary = log10(salary), # there's a reason for this, I promise
         lgID = factor(lgID, levels = c("AL", "NL"))) # fix the defunct league issue

set.seed(11249)
batting_2016_split <- initial_split(batting_2016, prop = 0.75)
batting_train <- training(batting_2016_split)
batting_test <- testing(batting_2016_split)
```

```
RMP <- read_csv("../Data Sets/RateMyProfessor.csv")
```

```
Rows: 22038 Columns: 15
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (7): hotness, rank, gender, race, discipline, uni_type, uni_control
dbl (6): id, overall, difficulty, review_count, interest, scientific_age
```

```
lgl (2): mentions_accent, mentions_ta

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
RMP$hotness <- factor(RMP$hotness, levels = c("hot", "cold"))
RMP$rank <- factor(RMP$rank,
                   levels = c("Assistant Professor", "Associate Professor", "Professor"),
                   labels = c("Asst", "Assoc", "Full"))
set.seed(1880)
RMP_split <- initial_split(RMP, prop = 0.80)

RMP_train <- training(RMP_split)
RMP_test <- testing(RMP_split)
```

Regression Trees

1. Explain how a decision tree model works.

When we create a decision tree, there are three parameters that we can tune:

- `tree_depth` indicates the maximum depth of the tree - the default is 30
- `min_n` determines whether a node is a terminal node (if there are fewer `min_n` observations at a node, then the node is a terminal node) - the default is 2
- `cost_complexity` indicates the cost-complexity parameter for tree pruning - the default is 0.01

Here we'll leave the first two parameters at their defaults and tune the cost-complexity parameter.

2. Explain the idea behind cost-complexity pruning.
3. Explain the tuning of each parameter in terms of the bias-variance tradeoff.

```
treeR_model <- decision_tree(
  mode = "regression", # classification or regression
  engine = "rpart", # use rpart package to fit the tree
  cost_complexity = tune() # tune this, but leave min_n and max_depth at their defaults
)
```

Pre-processing is a good idea but not strictly necessary here.

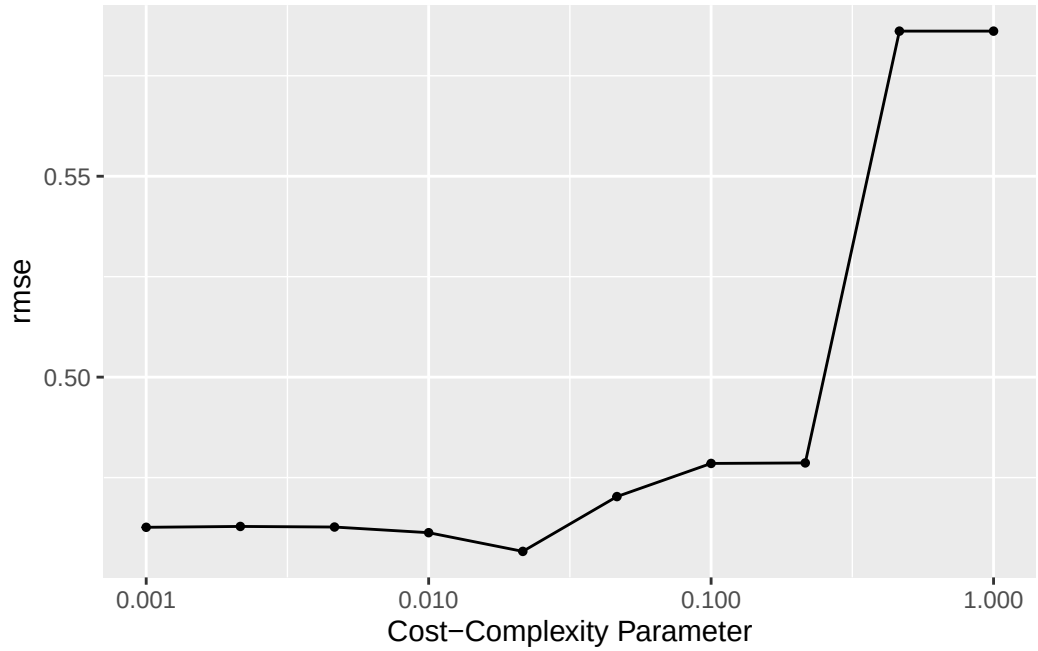
```
treeR_recipe <- recipe(
  salary ~ G +  R + H + X2B + X3B + HR + RBI +
    SB + CS + BB + SO + IBB + HBP + SH + SF + GIDP +
    age + lgID, # response ~ predictors
  data = batting_train
)

treeR_wflow <- workflow() |>
  add_model(treeR_model) |>
  add_recipe(treeR_recipe)
```

Now we can do the cross-validation. We'll use the `grid_regular` function to set up our grid again. If we wanted finer control over our grid, we'd use `expand.grid` to set up our grid manually.

```
set.seed(156)
batting_kfold <- vfold_cv(batting_train,
                          v = 5, repeats = 3)

treeR_tune <- tune_grid(
  treeR_wflow,
  resamples = batting_kfold,
  grid = grid_regular(
    cost_complexity(range = c(-3, 0)),
    levels = 10
  ),
  metrics = metric_set(rmse) # warning message when using R^2
)
```

Just like with penalized regression, the `range` argument is on the log10-scale; in other words, we are searching from 10^{-3} (very little pruning) to $10^0 = 1$.

```
autoplot(treeR_tune)
```



1. Suppose we are using the one-standard-error rule. Will smaller values of `cost_complexity` or larger values result in simpler models? Why?
2. In my comments I said that we get a warning using R-squared. Recall that we get this warning when every observation is predicted to have the same value. In that case, what would the tree look like?

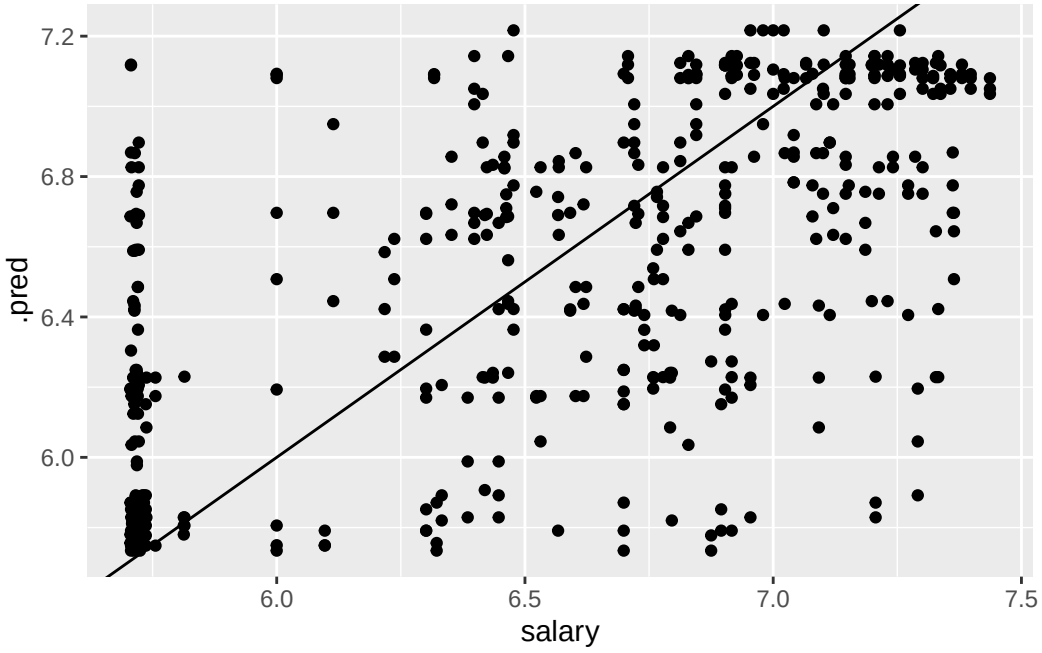
```
treeR_best <- select_by_one_std_err(
  treeR_tune,
  metric = "rmse",
  # how to sort based on tuning parameter(s)
  desc(cost_complexity)
)
treeR_best
```

```
# A tibble: 1 x 2
  cost_complexity .config
            <dbl> <chr>
1          0.0215 Preprocessor1_Model05
```

Now we finalize our workflow and look at how well calibrated the model is:

```
treeR_wflow_final <- treeR_wflow |>
  finalize_workflow(parameters = treeR_best)
```

```
# calibration: we want the predictions to be "on average" correct across the entire range of
# traditionally for regression: observed y on x-axis, predicted y on y-axis (ML doesn't abide
# we don't look at the test set, we use cross-validation
treeR_pred_check <- treeR_wflow_final |>
  fit_resamples(
    resamples = batting_kfold,
    control = control_resamples(
      save_pred = TRUE # store the predicted y-values
    )
  ) |>
  collect_predictions() # extract the predictions from the output

treeR_pred_check |>
  ggplot(
    mapping = aes(x = salary, y = .pred)
  ) +
  geom_point() +
  geom_abline(slope = 1, intercept = 0) # y = x line
```



Well, this certainly looks different from the linear regression models!

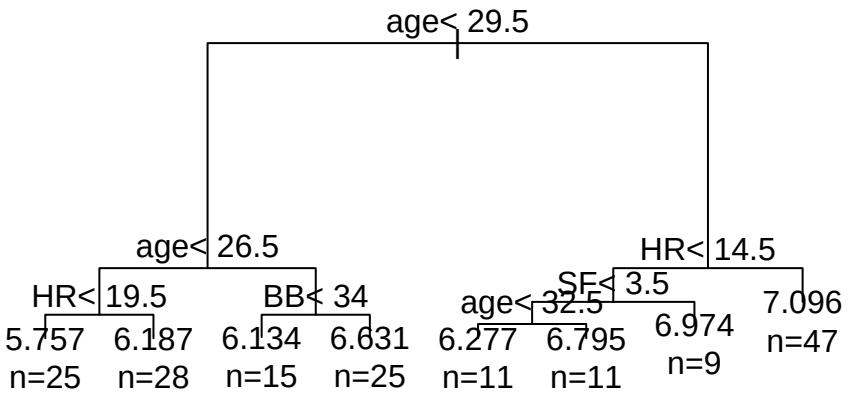
3. Explain why we see this pattern - a bunch of horizontal bands - instead.

Assuming we're okay with this somehow-worse-than-linear-regression calibration, we fit the model on the full training set:

```
treeR_fit <- fit(treeR_wflow_final,
                 data = batting_train)
```

and investigate what the model actually looks like:

```
par(xpd = TRUE) # makes it so that we can see the tree
treeR_fit |>
  extract_fit_engine() |>
  plot()
treeR_fit |>
  extract_fit_engine() |>
  text(use.n = TRUE)
```



3. Explain how to read this tree.

We then make predictions on the holdout set:

```
predictions_treeR <- broom::augment(treeR_fit, new_data = batting_test)
predictions_treeR |> dplyr::select(
  nameFirst, nameLast, age, HR, salary, .pred
)
```

```
# A tibble: 57 x 6
   nameFirst nameLast   age    HR salary .pred
   <chr>     <chr>    <int> <int>  <dbl> <dbl>
 1 Jose      Abreu       29    25   7.07  6.63
 2 Javier    Baez        23    14   5.71  5.76
 3 Darwin    Barney      30     4   6.02  6.28
 4 Mookie    Betts       23    31   5.75  6.19
 5 Charlie   Blackmon    30    29   6.54  7.10
 6 Peter     Bourjos     29     5   6.30  6.13
 7 Miguel    Cabrera     33    38   7.45  7.10
 8 Ezequiel  Carrera     29     6   5.72  6.13
 9 Jason     Castro      29    11   6.70  6.63
10 Francisco Cervelli    30     1   6.54  6.97
# i 47 more rows
```

4. Explain why Javier Baez and Mookie Betts are predicted to have the same salary, even though Baez hit fewer than 14.5 home runs and Betts hit more than 14.5 home runs.

Classification Trees

Because we already set up the decision tree model, we can just update it to do classification using `set_mode`:

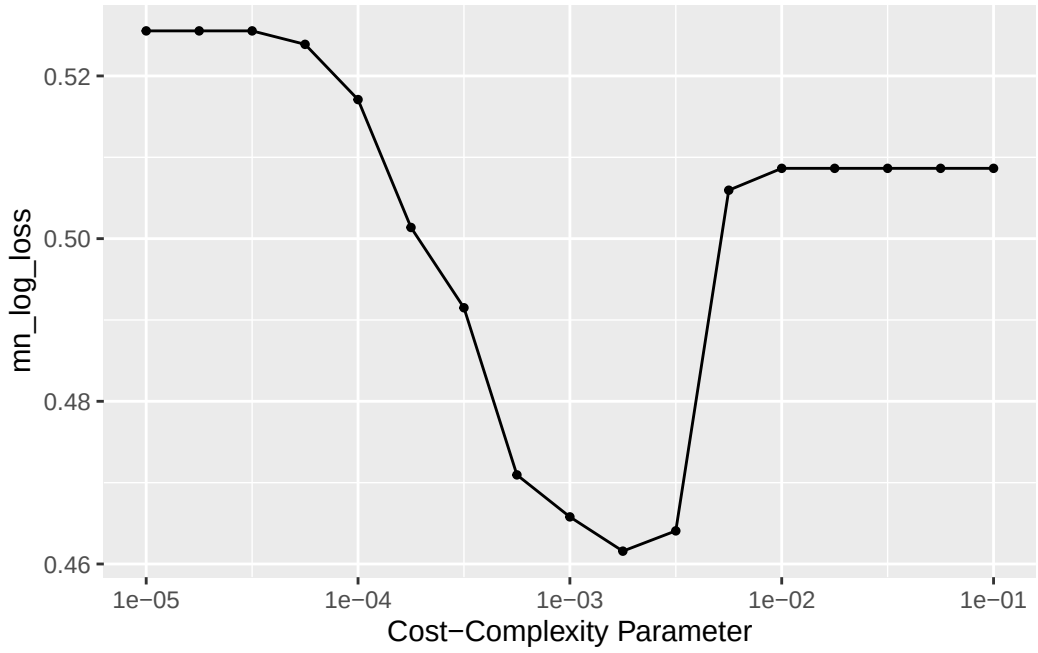
```
treeC_model <- treeR_model |>
  set_mode("classification")

# Equivalent to:
#treeC_model <- decision_tree(mode = "classification", engine = "rpart", cost_complexity = tu
# because we are still using rpart and tuning the cost-complexity parameter
```

Pre-processing is a good idea but not strictly necessary here. Remember that we are using entirely different data to do entirely different predictions, so we do have to set up the new recipe.

```
treeC_recipe <- recipe(
  hotness ~ scientific_age + gender + interest + uni_control + mentions_accent + discipline +
)

treeC_wflow <- workflow() |>
  add_model(treeC_model) |>
  add_recipe(treeC_recipe)
```

Now we can do the cross-validation. Traditionally, mean log-loss is used to choose the best decision tree model, so we'll look at that.

```
set.seed(1332)
RMP_kfold <- vfold_cv(RMP_train, v = 5, repeats = 3)
treeC_tune1 <- tune_grid(treeC_model,
                         treeC_recipe,
                         resamples = RMP_kfold,
                         metrics = metric_set(mn_log_loss),
                         grid = grid_regular(cost_complexity(range = c(-5, -1)), levels = 17)) #

autoplot(treeC_tune1)
```



1. Explain how to read this graph.

```r
treeC_best <- select_by_one_std_err(
  treeC_tune1,
  metric = "mn_log_loss",
  desc(cost_complexity)
)
treeC_best
```

```
# A tibble: 1 x 2
  cost_complexity .config
            <dbl> <chr>
1         0.00316 Preprocessor1_Model11
```

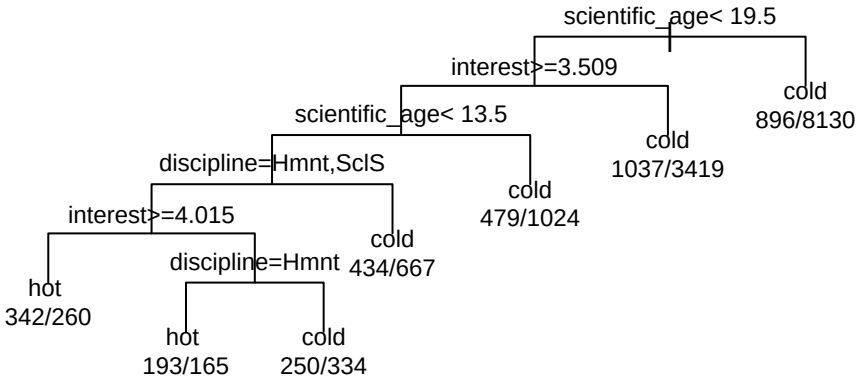
Now we finalize our workflow and fit the model on the training set:

```r
treeC_wflow_final <- finalize_workflow(
  treeC_wflow,
  parameters = treeC_best
  )

treeC_fit <- fit(treeC_wflow_final, data = RMP_train)
treeC_fit
```

```
== Workflow [trained] ==========================================================
Preprocessor: Recipe
Model: decision_tree()

-- Preprocessor ----------------------------------------------------------------
0 Recipe Steps

-- Model -----------------------------------------------------------------------
n= 17630

node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 17630 3631 cold (0.20595576 0.79404424)
   2) scientific_age< 19.5 8604 2735 cold (0.31787541 0.68212459)
     4) interest>=3.509259 4148 1698 cold (0.40935391 0.59064609)
       8) scientific_age< 13.5 2645 1219 cold (0.46086957 0.53913043)
        16) discipline=Humanities,Social Sciences 1544  759 hot (0.50841969 0.49158031)
          32) interest>=4.014706 602  260 hot (0.56810631 0.43189369) *
```

```
          33) interest< 4.014706 942  443 cold (0.47027601 0.52972399)
            66) discipline=Humanities 358  165 hot (0.53910615 0.46089385) *
            67) discipline=Social Sciences 584  250 cold (0.42808219 0.57191781) *
         17) discipline=Engineering,Medical sciences,Natural Sciences 1101  434 cold (0.39418
       9) scientific_age>=13.5 1503  479 cold (0.31869594 0.68130406) *
     5) interest< 3.509259 4456 1037 cold (0.23271993 0.76728007) *
   3) scientific_age>=19.5 9026  896 cold (0.09926878 0.90073122) *
```

```
par(xpd = TRUE) # otherwise things clip
extract_fit_engine(treeC_fit) |>
  plot(compress = TRUE, margin = 0, uniform = TRUE)
# uniform = TRUE makes the tree drawing more uniform
# helps to space out the text issues

extract_fit_engine(treeC_fit) |>
  text(use.n = TRUE, minlength = 4, cex = 0.75)
```




```
# cex = 0.75 puts text at 3/4 size for readability
```

2. Explain how to read this tree.

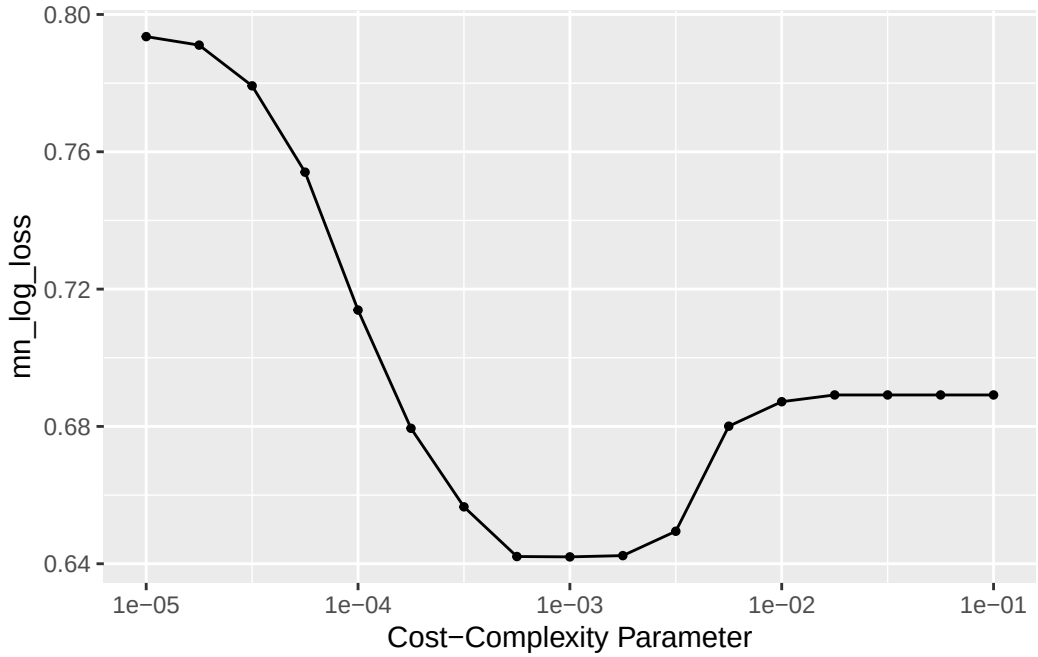
```
broom::augment(treeC_fit, new_data = RMP_test) |>
  conf_mat(truth = hotness, estimate = .pred_class)
```

```
          Truth
Prediction  hot cold
      hot   123   98
      cold  809 3378
```

Multiclass Classification

```
treeMC_recipe <- recipe(
  rank ~ scientific_age + gender + race + uni_control + mentions_accent + discipline + uni_ty
  data = RMP_train
)

treeMC_wflow <- workflow() |>
  add_model(treeC_model) |>
  add_recipe(treeMC_recipe)
```

```
treeMC_tune <- tune_grid(treeC_model,
                         treeMC_recipe,
                         resamples = RMP_kfold,
                         metrics = metric_set(mn_log_loss),
                         grid = grid_regular(cost_complexity(range = c(-5, -1)), levels = 17))

autoplot(treeMC_tune)
```

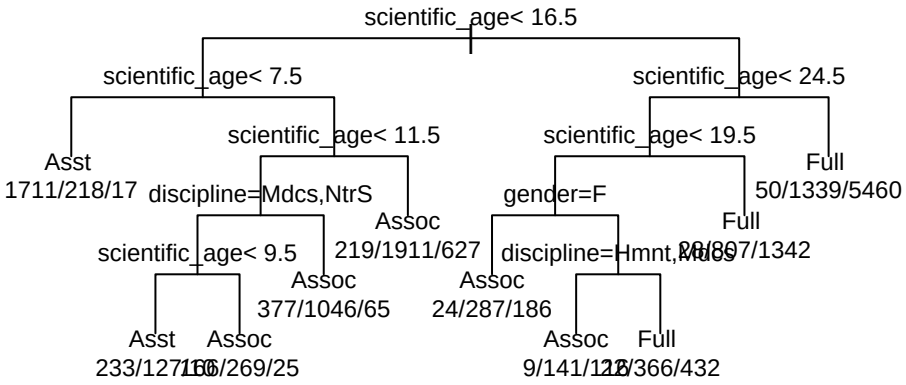



```
treeMC_best <- select_by_one_std_err(
  treeMC_tune,
  metric = "mn_log_loss",
  desc(cost_complexity)
)
treeMC_best
```

```
# A tibble: 1 x 2
  cost_complexity .config
            <dbl> <chr>
1         0.00178 Preprocessor1_Model10
```

```
treeMC_wflow_final <- finalize_workflow(
  treeMC_wflow,
  parameters = treeMC_best
  )

treeMC_fit <- fit(treeMC_wflow_final,
                  data = RMP_train)

par(xpd = TRUE)
extract_fit_engine(treeMC_fit) |>
```

```
  plot(uniform = TRUE)
extract_fit_engine(treeMC_fit) |>
  text(use.n = TRUE, minlength = 4, cex = 0.75)
```



3. Explain how to read this tree. Why are there so many `scientific_age` splits?